

RUNNING HEAD: LSC OFFICIAL WEBSITE AND ADMINISTRATION SYSTEM SRS

SOFTWARE REQUIREMENTS SPECIFICATION

**Lideta Sub-City Official Website
and Administration System**

*A technical specification for system architecture, functional requirements,
security requirements, API contracts, and deployment infrastructure*

Document Number:	LSC-SRS-001
Version:	1.0.0 — Final
Date of Issue:	August 2026
Prepared By:	Development Team, Lideta Sub-City Administration IT Department
Reviewed By:	Information Network Security Administration
Approved By:	Lideta Sub-City Administration
Standard:	IEEE Std 830-1998 / APA Publication Manual, 7th Edition

CONFIDENTIAL — FOR INTERNAL USE ONLY

Unauthorized reproduction or distribution of this document is strictly prohibited.

Table of Contents

1. Introduction	3
1.1 Purpose	3
1.2 Scope	3
1.3 Definitions, Acronyms, and Abbreviations	4
1.4 References	5
1.5 Overview	5
2. Overall Description	6
2.1 Product Perspective	6
2.2 Product Functions Summary	6
2.3 User Classes and Characteristics	7
2.4 Operating Environment	7
2.5 Design and Implementation Constraints	8
2.6 Assumptions and Dependencies	8
3. System Architecture	9
3.1 Architectural Pattern	9
3.2 Front-End Architecture	10
3.3 Back-End Architecture	11
3.4 Data Flow Overview	12
4. Roles and Access Control	13
5. Functional Requirements	16
6. Non-Functional Requirements	26
7. Database Schema	28
8. API Specification	34
9. Real-Time Communication	43
10. Security Requirements	44
11. Deployment Architecture	49
12. Appendices	54
Appendix A Technology Stack Summary	54
Appendix B Complete Route Tree	55
Appendix C File Upload Paths	56
Appendix D Environment Variable Reference	56
Appendix E Admin Role Default Landing Paths	57
Appendix F Satisfaction Survey Question Reference	57
Appendix G Known Limitations and Future Recommendations	58

Software Requirements Specification

Lideta Sub-City Official Website and Administration System

Version 1.0.0 | Confidential

Document Control

Field	Value
Project Name	Lideta Sub-City Official Website and Administration System
Document Type	Software Requirements Specification (SRS)
Version	1.0.0
Status	Final
Prepared For	IT Department / Information Network Security Administration
Classification	Confidential — Internal Use Only
Date	August 2026

Table of Contents

1. [Introduction](#)
2. [Overall Description](#)
3. [System Architecture](#)
4. [Roles and Access Control](#)
5. [Functional Requirements](#)
6. [Non-Functional Requirements](#)
7. [Database Schema](#)
8. [API Specification](#)
9. [Real-Time Communication](#)
10. [Security Requirements](#)
11. [Deployment Architecture](#)
12. [Appendices](#)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for the Lideta Sub-City Official Website and Administration System. It is intended to serve two primary audiences:

1. **IT Department Developers** — as a technical reference for understanding, maintaining, and extending the system.
2. **Information Network Security Administration** — as a comprehensive baseline document for security analysis, vulnerability assessment, and penetration testing of all system components.

This document covers the public-facing citizen portal, the administrative back-office, the RESTful API layer, the database design, access control policies, and the deployment infrastructure.

1.2 Scope

The system is a full-stack web application that serves as the official digital platform for Lideta Sub-City Administration, Addis Ababa, Ethiopia. It provides:

- A **public citizen portal** for accessing government news, events, department information, job vacancies, and submitting complaints and contact inquiries.
- A **citizen account system** allowing registered users to track complaints and job applications.
- A **multi-role administration panel** for government staff to manage all content and citizen interactions.
- A **super-administration panel** for account management and system-wide oversight.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
OTP	One-Time Password
CRUD	Create, Read, Update, Delete
SPA	Single-Page Application
RBAC	Role-Based Access Control
ESM	ECMAScript Modules
CORS	Cross-Origin Resource Sharing
CDN	Content Delivery Network
SSL/TLS	Secure Sockets Layer / Transport Layer Security
DFD	Data Flow Diagram
ER	Entity-Relationship
VITE	Next-generation front-end build tool
CV	Curriculum Vitae
2FA	Two-Factor Authentication

1.4 References

- Express.js v5 Documentation — <https://expressjs.com>
- React v19 Documentation — <https://react.dev>
- React Router v7 Documentation — <https://reactrouter.com>
- JSON Web Tokens RFC 7519 — <https://tools.ietf.org/html/rfc7519>
- Cloud OTP Email Service Provider Documentation
- OWASP Top 10 Web Application Security Risks — <https://owasp.org/Top10>
- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications

1.5 Overview

This document is structured as follows: Section 2 describes the overall system context and constraints. Section 3 details the system architecture. Section 4 defines all roles and their access permissions. Sections 5 and 6 specify functional and non-functional requirements respectively. Section 7 presents the database schema. Section 8 provides the full API specification.

Section 9 addresses real-time communication. Section 10 defines security requirements. Section 11 describes the deployment architecture. Section 12 contains appendices including diagrams.

2. Overall Description

2.1 Product Perspective

The Lideta Sub-City Official Website and Administration System is a greenfield web application developed for Lideta Sub-City Administration, one of the ten sub-cities of Addis Ababa, Ethiopia. It replaces a static informational web presence with a dynamic, database-driven platform that enables two-way interaction between the government and its citizens.

The system operates as a monolithic full-stack application in which a single Node.js/Express server serves both the REST API and the compiled React SPA. It integrates with a cloud OTP email service exclusively for OTP email delivery, while all persistent application data resides in a MySQL database hosted on Plesk.

2.2 Product Functions Summary

The system performs the following high-level functions:

- **Content Publication** — Publishing and managing news articles, events, and department information in three languages (English, Amharic, Oromo).
- **Citizen Services** — Accepting public complaints, contact inquiries, and service satisfaction surveys.
- **Career Portal** — Posting job vacancies and processing online applications including CV file uploads.
- **Citizen Accounts** — User registration, authentication, and a personal dashboard for tracking complaints and applications.
- **Administration** — Role-segmented back-office for staff to manage all content and citizen interactions.
- **Super-Administration** — System-wide account management and operational oversight.

2.3 User Classes and Characteristics

User Class	Description	Technical Proficiency
Anonymous Visitor	Any person accessing the public site without an account	Non-technical; general public
Registered Citizen	A verified user with an account; can track complaints and applications	Non-technical; general public
Complaint Admin	Staff member managing citizen complaints	Low-to-medium; office worker
Event Admin	Staff member managing event publications	Low-to-medium; office worker
News Admin	Staff member managing news articles	Low-to-medium; office writer
Vacancy Admin	Staff member managing job postings and applicants	Low-to-medium; HR staff
General Admin	Staff member with access to the main dashboard and all content	Medium; department head
Super Admin	System administrator with full access including account management	High; IT staff

2.4 Operating Environment

- **Client:** Any modern web browser (Chrome, Firefox, Safari, Edge) on desktop or mobile. No native application is required.
- **Server:** Node.js v18+ runtime on Plesk Obsidian shared web hosting (EthioTelecom infrastructure), running on Linux.
- **Database:** MySQL (Plesk built-in) for all application data.
- **Email/OTP:** Cloud email delivery service for OTP dispatch.
- **File Storage:** Local filesystem on the Plesk server at `client/public/uploads/`.

2.5 Design and Implementation Constraints

- The system must support three languages: English (en), Amharic (am), and Oromo (or) on all public-facing content.
- All API endpoints must use HTTPS in production.
- File uploads are constrained to 50MB per file.
- The application is deployed as a single server process; horizontal scaling is not currently supported.
- The front-end is a Single-Page Application served from the same Express server that handles API requests.
- JWT tokens are used for both admin and citizen authentication, stored in localStorage on the client side.

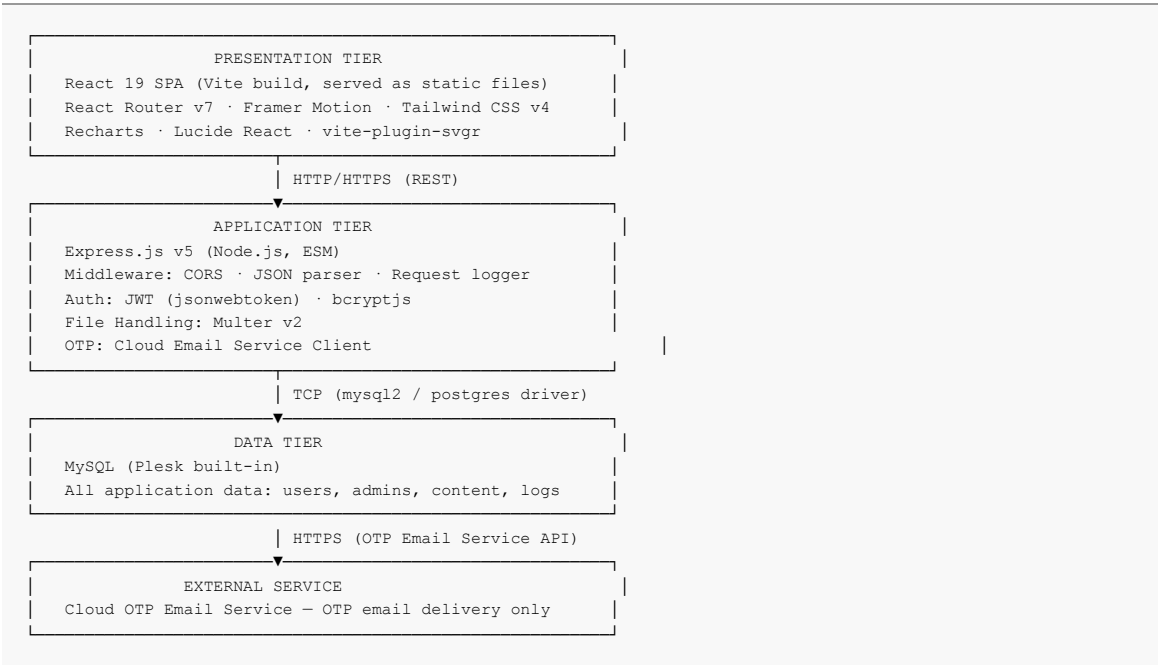
2.6 Assumptions and Dependencies

- The cloud email delivery service is available and reachable. If it is unreachable, OTP-dependent flows (email verification, 2FA) will fail.
- The Plesk server has Node.js Passenger module enabled.
- The MySQL database is provisioned and accessible to the Node.js process via the connection string in environment variables.
- The client/dist directory contains a valid React production build before the server is started.
- SMTP/email delivery SLA is governed by the email service provider's infrastructure and is outside the control of this application.

3. System Architecture

3.1 Architectural Pattern

The system follows a **monolithic three-tier architecture**:



3.2 Front-End Architecture

The client is a React 19 Single-Page Application built with Vite 7.

Key architectural decisions:

- **Routing:** React Router v7 with nested layouts. The `Layout` component is the public layout shell; `AdminLayout` is the admin layout shell; `SuperAdminLayout` is the superadmin shell.
- **State Management:** No external state management library. State is managed via React Context API:
- `LanguageContext` — global language selection (en/am/or), persisted in `localStorage`.
- `UserContext` — citizen authentication state, token from `localStorage`.
- `adminContext` — admin authentication state, provided by layout components not a standalone provider.
- **Styling:** Tailwind CSS v4 (CSS-first configuration via Vite plugin). Custom fonts (Goldman, Jost, Roboto) loaded via `@font-face`. Custom animation keyframes defined in `index.css`.
- **Animation:** Framer Motion v13 for page transitions, stagger animations, and `AnimatePresence` for route changes.
- **Internationalization:** Manual i18n via a static `translated_contents.json` file. All UI strings for public pages are keyed by `{ en, am, or }`. Dynamic content (news, events, etc.) uses `amh` and `orm` JSONB columns from the database.
- **SVG Icons:** All icons are SVG files imported as React components via `vite-plugin-svgr`.
- **Charts:** Recharts v3 for the admin dashboard complaint statistics charts.

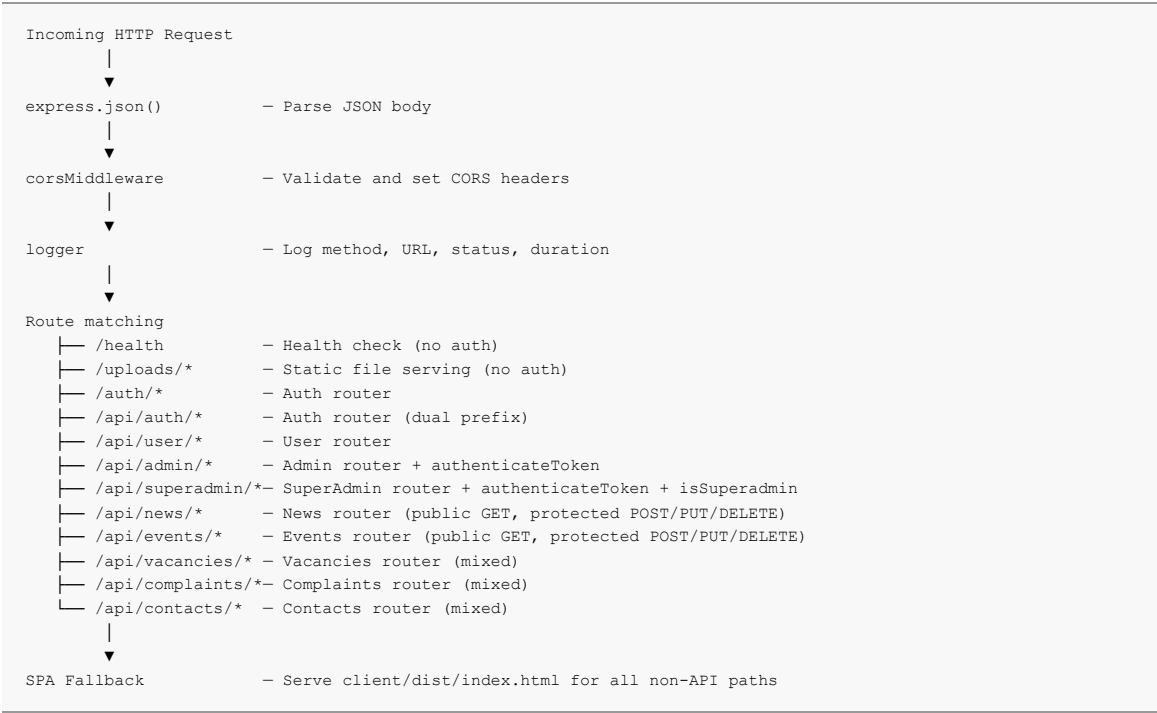
Client directory structure:

```
client/src/
├─ App.jsx           ─ Root router and layout composition
├─ index.css         ─ Tailwind imports, custom keyframes, font-face declarations
├─ assets/           ─ Static assets (images, SVG icons, fonts)
│  └─ icons/         ─ 73 SVG icon files
│  └─ fonts/         ─ Goldman, Jost, Roboto font files
├─ components/
│  └─ shared/        ─ Navbar, Footer, AdminTop, Sidebar
│  └─ ui/            ─ Reusable UI components (cards, modals, inputs)
│  └─ forms/         ─ GlobalSatisfactionTrigger, ServiceSatisfactionForm
│  └─ utils/         ─ Context providers, guards, hooks
├─ data/             ─ Static JSON data files and department assets
├─ hooks/            ─ useInView custom hook
├─ pages/            ─ All page components
│  └─ Admin/         ─ 8 admin panel pages
│  └─ SuperAdmin/    ─ 3 superadmin pages
└─ utils/            ─ api.js, passwordHelper.js, validation.js, roleLabels.js
```

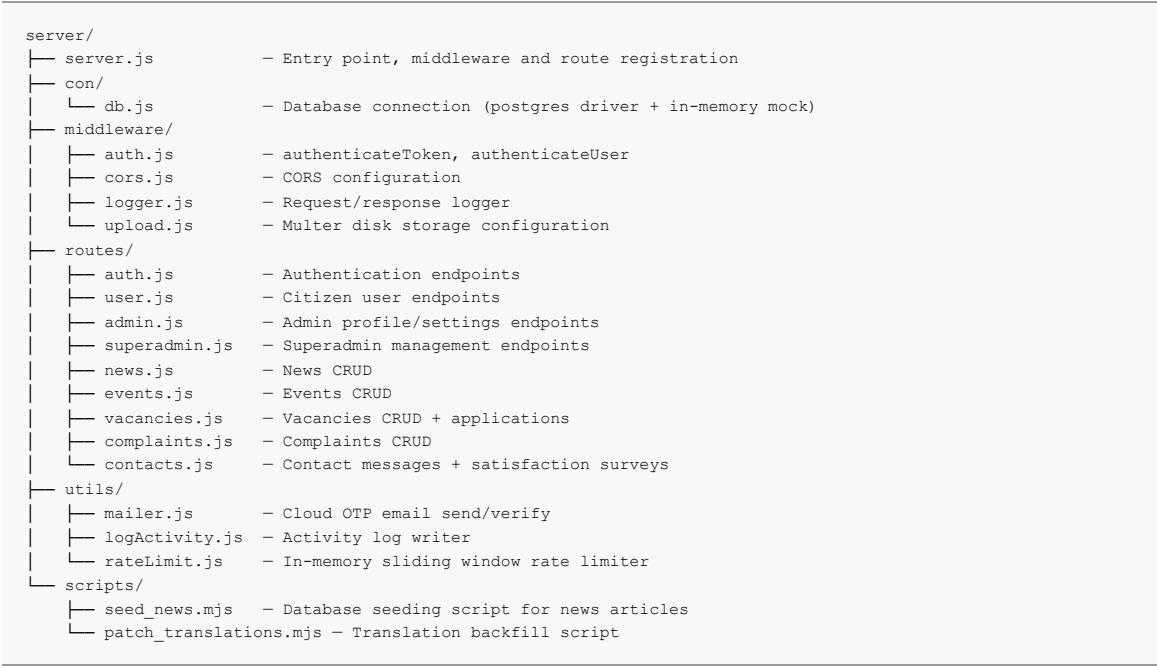
3.3 Back-End Architecture

The server is an Express.js v5 application running as a single Node.js process in ESM mode.

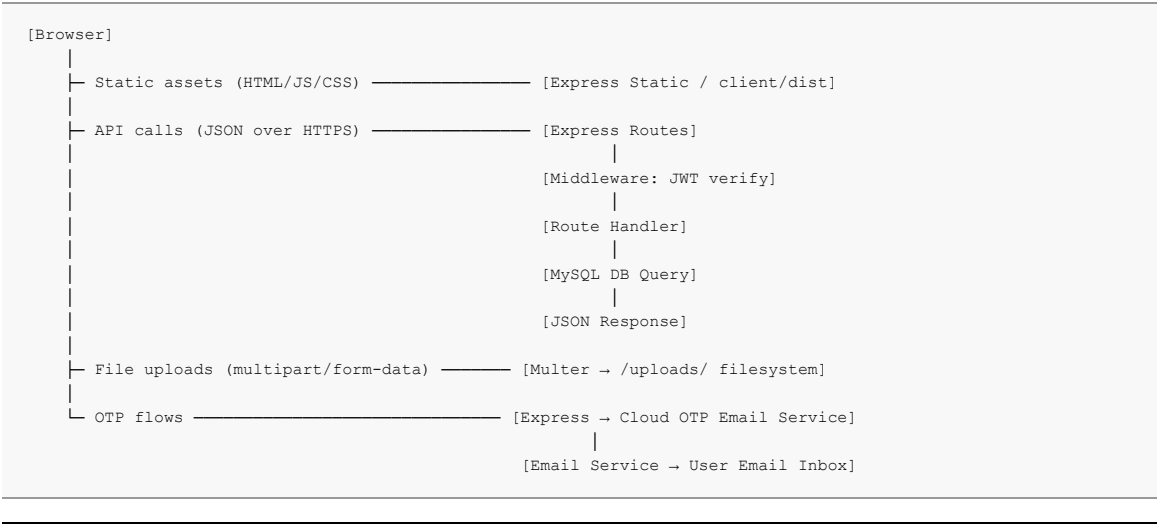
Request processing pipeline:



Server directory structure:



3.4 Data Flow Overview



4. Roles and Access Control

4.1 Role Overview

The system implements Role-Based Access Control (RBAC) with seven distinct roles. Roles are stored in the `admins.role` column. Citizen users are a separate entity stored in the `users` table and are not part of the admin RBAC system.

4.2 Role Definitions

Role Key	Display Name	Scope
<code>superadmin</code>	Super Admin	Full system access; account management
<code>admin</code>	General Admin	Dashboard + all content management
<code>complaint_admin</code>	Complaint Admin	Complaints module only
<code>event_admin</code>	Event Admin	Events module only
<code>news_admin</code>	News Admin	News module only
<code>vacancy_admin</code>	Vacancy Admin	Vacancies and applicants module only
<code>(citizen)</code>	Registered User	Complaint submission and application tracking via <code>/account</code>

4.3 Permission Matrix

Feature / Route	Super Admin	General Admin	Complaint Admin	Event Admin	News Admin	Vacancy Admin	Citizen	Anonymous
View public website	✓	✓	✓	✓	✓	✓	✓	✓
Submit complaint	✓	✓	✓	✓	✓	✓	✓	✓
Submit contact form	✓	✓	✓	✓	✓	✓	✓	✓
Apply for vacancy	✓	✓	✓	✓	✓	✓	✓	—
View own dashboard	—	—	—	—	—	—	✓	—
Admin dashboard (/admin)	✓	✓	—	—	—	—	—	—
Manage complaints	✓	✓	✓	—	—	—	—	—
Manage events	✓	✓	—	✓	—	—	—	—
Manage news	✓	✓	—	—	✓	—	—	—
Manage vacancies + applicants	✓	✓	—	—	—	✓	—	—
View admin profile	✓	✓	✓	✓	✓	✓	—	—
Create peer admin account	✓	✓	—	—	—	—	—	—
View all admin accounts	✓	—	—	—	—	—	—	—
Create/edit/delete any admin	✓	—	—	—	—	—	—	—
View system overview stats	✓	—	—	—	—	—	—	—
View all vacancy applications	✓	—	—	—	—	—	—	—

4.4 Role Enforcement

Server-side (authoritative):

- `authenticateToken` middleware verifies the admin JWT on all protected routes and attaches `req.admin`.
- `isSuperadmin()` check in `superadmin.js` routes returns HTTP 403 if `req.admin.role !== 'superadmin'`.
- Admin routes do not restrict by specific sub-role on the API level (any authenticated admin can call most `/api/admin/*` endpoints); role restriction is enforced at the route-guard level on the client.

Client-side (UI enforcement, not a security boundary):

- `RoleGuard` component reads `adminContext.admin.role` and redirects to the role's default landing page if the current route is not permitted.
- Superadmin bypasses all `RoleGuard` checks.
- Role default landing paths: `admin` → `/admin`, `complaintadmin` → `/admin/complaints`, `event admin` → `/admin/events`, `newsadmin` → `/admin/news`, `vacancy admin` → `/admin/vacancy`, `superadmin` → `/superadmin/home`.
- The admin sidebar renders navigation items conditionally based on the logged-in role.

4.5 Token Management

Token Type	Storage	Expiry	Key
Admin JWT	<code>localStorage.getItem('token')</code>	7 days	<code>token</code>
Citizen JWT	<code>localStorage.getItem('userToken')</code>	30 days	<code>userToken</code>

Both tokens share the same `JWT_SECRET` environment variable but are issued by different routes (`/auth/admin/login` vs `/api/auth/user/login`) and verified by different middleware (`authenticateToken` vs `authenticateUser`).

5. Functional Requirements

5.1 Public Portal

5.1.1 Language Switching

- **FR-001:** The system shall support three display languages: English (en), Amharic (am), and Oromo (or).
- **FR-002:** Users shall be able to switch language at any time using the navbar language selector.
- **FR-003:** The selected language shall be persisted in `localStorage` and restored on subsequent visits.
- **FR-004:** All dynamic content (news, events, vacancies, departments) shall display in the selected language if a translation exists, falling back to English if not.

5.1.2 Home Page

- **FR-005:** The home page shall display a hero section, a latest news preview (most recent 3 articles), additional services section (events, news, jobs), and a complaints call-to-action.
- **FR-006:** The home page shall be fully responsive across mobile, tablet, and desktop viewports.

5.1.3 Departments

- **FR-007:** The system shall display all 19 sub-city departments grouped by category.
- **FR-008:** Departments shall be filterable by category via a horizontal tab bar.
- **FR-009:** Each department card shall display the department logo (or an initial avatar if no logo exists), name, short description, and category.
- **FR-010:** Clicking a department card shall navigate to a detail page showing: full name, leader name, head photo, mission statement, vision, services list, core values, and contact information (location, email, phone, office hours, website).
- **FR-011:** Department data is static (loaded from `departments.json`) and does not require a database call.

5.1.4 News

- **FR-012:** The news listing page shall display articles in a BBC/NYT-inspired editorial layout: a full-width hero article, a two-article feature row, and a horizontal list of remaining articles.
- **FR-013:** The page shall include a breaking news banner showing the current date and total article count.
- **FR-014:** Articles shall be filterable by category via a horizontal tab bar (Technology, Infrastructure, Health, Education, Events, Security, Environment).
- **FR-015:** A search input shall filter articles in real time across title and description fields in all three languages.
- **FR-016:** The news detail page shall display: category pill, date, reading time estimate, headline, hero image with blur-up loading, article body with amber lede treatment, social share buttons (Facebook, Telegram, Twitter, Instagram, Email, Copy Link), and a "More from this category" section.
- **FR-017:** The detail page shall include a sticky right sidebar with "Latest News" items.

- **FR-018:** An amber reading progress bar shall be shown fixed at the top of the viewport during article reading.

5.1.5 Events

- **FR-019:** The events listing page shall display events grouped by month with a calendar-widget style card showing the event's month header (colored by status) and day number.
- **FR-020:** Events shall be filterable by status (All, Upcoming, Pending, Complete, Canceled) via a tab bar.
- **FR-021:** A search input shall filter events by title and location.
- **FR-022:** The event detail page shall display: status pill, date, location, event title, a hero image with the calendar widget overlaid bottom-left, event description with lede treatment, and a right sidebar with related events.

5.1.6 Vacancies (Career Portal)

- **FR-023:** The vacancy listing page shall default to a category directory view showing all job categories as clickable rows with job counts, matching the EthioJobs-style design.
- **FR-024:** Clicking a category shall transition to a filtered job list view with a left sidebar for additional filtering.
- **FR-025:** The sidebar shall support filtering by: search term, category, and job type (Full Time, Part Time, Contract, Internship, Remote).
- **FR-026:** The sidebar shall be a slide-in off-canvas drawer on mobile and a sticky column on desktop.
- **FR-027:** Each vacancy card shall display: job title, category, employment type, location, and posting date. Salary shall NOT be displayed.
- **FR-028:** The vacancy detail page shall display the full job description, responsibilities, qualifications, and required skills.
- **FR-029:** Unauthenticated users shall see an authentication overlay on the application sidebar and be redirected to `/account/auth?next=/vacancy/:id` to sign in before applying.
- **FR-030:** Authenticated users shall be able to apply by submitting their name, email, phone, and a CV file (PDF only, maximum 30MB).
- **FR-031:** Upon successful application, the system shall return an application reference number in the format `APP-XXXXX`.

5.1.7 Complaints

- **FR-032:** Any visitor (authenticated or not) shall be able to submit a complaint.
- **FR-033:** The complaint form shall collect: full name, address, sub-city, woreda, complaint type, description, and optionally media attachments (photos, videos, audio recordings).
- **FR-034:** The system shall support in-browser audio and video recording via the MediaRecorder API for complaint attachments.
- **FR-035:** The system shall return a complaint reference number in the format `CPL-XXXXX` upon successful submission.
- **FR-036:** Authenticated citizens shall be able to view their submitted complaints and their current status from the `/account` dashboard.

5.1.8 Contacts

- **FR-037:** Any visitor shall be able to submit a contact message with their name, email, and message.
- **FR-038:** The system shall record all contact submissions in the database for admin review.

5.1.9 Service Satisfaction Survey

- **FR-039:** A floating `GlobalSatisfactionTrigger` widget shall appear on the public site after a configurable delay, prompting users to complete a satisfaction survey.
- **FR-040:** The survey shall collect demographic information (gender, age, marital status, education level, employment status, district, number of visits, services requested) and 11 Likert-scale satisfaction questions.
- **FR-041:** Survey responses shall be stored in the database and aggregated for admin reporting.

5.2 Citizen Account System

5.2.1 Registration and Login

- **FR-042:** Citizens shall register with: first name, last name, email, optional Ethiopian phone number, and a password meeting strength requirements.
- **FR-043:** Password strength requirements: minimum 8 characters, at least one uppercase letter, one lowercase letter, one digit, and one special character.
- **FR-044:** Ethiopian phone numbers shall be validated against the pattern: `09XXXXXXXX`, `07XXXXXXXX`, `+2519XXXXXXXX`, or `+2517XXXXXXXX`.
- **FR-045:** Upon registration, the system shall send a 6-digit OTP to the registered email address via the system's cloud email service for email verification.
- **FR-046:** Citizens shall log in using email and password.
- **FR-047:** The system shall support a forgot-password flow: submit email → receive OTP → enter OTP → set new password meeting strength requirements.
- **FR-048:** Reset passwords shall be subject to the same strength requirements as registration passwords.

5.2.2 Citizen Dashboard

- **FR-049:** The `/account` dashboard shall display the citizen's submitted complaints with their current status.
- **FR-050:** The dashboard shall display the citizen's job applications with the vacancy title and application status.
- **FR-051:** Citizens shall be able to update their profile (name, phone).

5.3 Administration Panel

5.3.1 Admin Authentication

- **FR-052:** Admins shall log in at `/auth/login` using a username and password.
- **FR-053:** The login system shall support an optional two-factor authentication (2FA) flow via OTP email if `requires2FA` is enabled for the account (infrastructure ready, flag currently set to `false` globally).
- **FR-054:** Admins shall be able to reset their password via the forgot-password OTP flow.

5.3.2 News Management

- **FR-055:** Authorized admins shall be able to create, edit, and delete news articles.
- **FR-056:** Each news article shall have: title, short description, full description, category, cover image, and translations for Amharic and Oromo.
- **FR-057:** The admin panel shall display a searchable, filterable list of all news articles with inline edit and delete actions.

5.3.3 Events Management

- **FR-058:** Authorized admins shall be able to create, edit, and delete events.

- **FR-059:** Each event shall have: title, description, location, start date, end date, status, photo, and Amharic/Oromo translations.
- **FR-060:** Admins shall be able to update an event's status (upcoming, pending, complete, canceled).

5.3.4 Vacancies Management

- **FR-061:** Authorized admins shall be able to create, edit, and delete job vacancies.
- **FR-062:** Each vacancy shall have: title, short description, full description, location, salary, type, category, skills (array), responsibilities (array), qualifications (array), start date, end date, and Amharic/Oromo translations.
- **FR-063:** Admins shall be able to view all applicants for each vacancy, including their submitted CV file.
- **FR-064:** Admins shall be able to update an applicant's status (submitted, reviewing, accepted, rejected).

5.3.5 Complaints Management

- **FR-065:** Authorized admins shall be able to view all complaints with filtering by status and type.
- **FR-066:** Admins shall be able to update complaint status, add notes, and assign a concerned staff member.
- **FR-067:** Admins shall be able to delete complaints.
- **FR-068:** The admin dashboard shall display complaint counts by status and a statistical chart.

5.3.6 Contact Management

- **FR-069:** Authorized admins shall be able to view pending contact messages.
- **FR-070:** Admins shall be able to mark contact messages as resolved.
- **FR-071:** The satisfaction survey statistics page shall display 30-day averages per question and daily response trends.

5.3.7 Admin Profile

- **FR-072:** All admins shall be able to update their personal information (name, gender, residency, phone, email).
- **FR-073:** All admins shall be able to change their password after verifying their current password.
- **FR-074:** All admins shall be able to upload and remove their profile picture.
- **FR-075:** Admins shall be able to configure their panel preferences: theme, font size, and language.

5.4 Super-Administration

- **FR-076:** The superadmin shall be able to view all admin accounts grouped by role.
- **FR-077:** The superadmin shall be able to create new admin accounts with any role from: Complaint Admin, Event Admin, News Admin, Vacancy Admin, Super Admin.
- **FR-078:** The superadmin shall be able to edit any admin account's details and role.
- **FR-079:** The superadmin shall be able to delete any admin account except their own.
- **FR-080:** The superadmin shall have access to a system overview with total complaints, resolved complaints, pending applications, and active events.
- **FR-081:** The superadmin shall be able to view all vacancy applicants across all postings with aggregated category statistics.
- **FR-082:** All admin management actions (create, update, delete) shall be recorded in the activity log.

6. Non-Functional Requirements

6.1 Performance

- **NFR-001:** The public home page shall achieve a Largest Contentful Paint (LCP) of under 2.5 seconds on a standard broadband connection.
- **NFR-002:** All REST API endpoints shall respond within 500ms under normal load (single concurrent user on shared hosting).
- **NFR-003:** The React SPA bundle shall be code-split and optimized by Vite; the main JS chunk shall not exceed 500KB gzipped.
- **NFR-004:** Database queries shall use indexed lookups where applicable; full-table scans shall be avoided for high-frequency endpoints (news GET, events GET, vacancies GET).

6.2 Availability

- **NFR-005:** The system shall maintain availability during routine Plesk maintenance windows by leveraging Phusion Passenger's automatic process management.
- **NFR-006:** The system shall gracefully handle database connection failures by returning HTTP 500 with a JSON error body rather than crashing the process.

6.3 Scalability

- **NFR-007:** The current monolithic deployment is designed for single-instance operation appropriate for sub-city government load volumes. The API layer is stateless (JWT-based auth) allowing future migration to a multi-instance deployment without architectural changes.

6.4 Usability

- **NFR-008:** All public-facing pages shall be fully responsive and usable on screens from 320px to 2560px wide.
- **NFR-009:** The system shall support three languages (EN/AM/OR) on all public pages. Language switching shall take effect immediately without a page reload.
- **NFR-010:** Form validation errors shall be displayed inline, below the relevant field, and must not rely solely on color to convey meaning.
- **NFR-011:** All interactive elements (buttons, links, form inputs) shall have accessible focus states and ARIA labels where appropriate.

6.5 Maintainability

- **NFR-012:** All server routes shall be organized in separate files by domain (news, events, vacancies, complaints, contacts, auth, user, admin, superadmin).
- **NFR-013:** All client-side form validation logic shall be centralized in `client/src/utls/validation.js`.
- **NFR-014:** Password strength validation shall be centralized in `client/src/utls/passwordHelper.js` and reused across all forms (registration, create admin, change password, reset password).
- **NFR-015:** Environment-specific configuration (database URL, JWT secret, email service keys) shall be managed exclusively via environment variables and never hard-coded in source files.

6.6 Reliability

- **NFR-016:** The database connection layer (`server/con/db.js`) shall fall back to an in-memory mock database in development environments where `DATABASE_URL` is not set, ensuring the server always starts.

- **NFR-017:** File upload operations shall be atomic: files shall be renamed to their final name only after the database record is confirmed.
- **NFR-018:** The OTP email system shall implement a 3.5-second retry mechanism when the email delivery service returns a rate-limit error.

6.7 Portability

- **NFR-019:** The application shall run on any Node.js v18+ environment with access to a MySQL database.
- **NFR-020:** The front-end build output (`client/dist`) shall be deployable to any static file hosting or CDN as a standalone SPA if the API base URL is configured via `VITEAPIURL`.

6.8 Internationalisation

- **NFR-021:** All translatable UI strings for public pages shall be defined in `translated_contents.json` with `en`, `am`, and `or` keys.
- **NFR-022:** Dynamic content stored in the database (news, events, vacancies) shall store Amharic and Oromo translations in JSONB-compatible columns (`amh`, `orm`) in the respective translation tables.
- **NFR-023:** The system shall fall back to the English version of any content where a translation in the requested language does not exist.

7. Database Schema

7.1 Overview

The database is MySQL, hosted on Plesk's built-in database server. The schema consists of 14 tables organized into four domains: authentication, content management, citizen interaction, and auditing.

7.2 Entity-Relationship Diagram (Textual)


```

ADMINS
| admin_id (PK)
| first_name, last_name, username (UNIQUE), email (UNIQUE)
| password_hash, role, gender, residency, phone_number, photo
| created_at
|
|< ADMIN_SETTINGS (1:1)
|     admin_id (PK, FK → admins)
|     theme, font_size, language
|
|< ACTIVITY_LOGS (1:N)
|     log_id (PK)
|     admin_id (FK → admins)
|     username, action, entity_type, entity_title
|     details (JSON), created_at

USERS
| id (PK)
| first_name, last_name, email (UNIQUE), phone
| password_hash, email_verified
| created_at
|
|< COMPLAINTS (1:N via user_id nullable)
|< APPLICANTS (1:N via user_id nullable)

NEWS
| id (PK)
| title, short_description, description
| category, photo (JSON)
| created_at
|
|< NEWS_TRANSLATION (1:1)
|     id (PK)
|     news_id (FK → news)
|     amh (JSON: {title, short_description, description, category})
|     orm (JSON: {title, short_description, description, category})

EVENTS
| events_id (PK)
| title, description, location
| start_date, end_date
| status (upcoming|pending|complete|canceled)
| photos (JSON array)
| created_at
|
|< EVENTS_TRANSLATION (1:1)
|     id (PK)
|     event_id (FK → events.events_id)
|     amh (JSON: {title, description, location})
|     orm (JSON: {title, description, location})

VACANCIES
| id (PK)
| title, short_description, description
| location, salary, type, category
| skills (JSON array), responsibilities (JSON array), qualifications (JSON array)
| start_date, end_date
| created_at
|
|< VACANCY_TRANSLATION (1:1)
|     id (PK)
|     vacancy_id (FK → vacancies)
|     amh (JSON: {title, short_description, description, category,
|         skills[], responsibilities[], qualifications[]})
|     orm (JSON: same structure)
|
|< APPLICANTS (1:N)
|     id (PK)
|     vacancy_id (FK → vacancies)
|     user_id (FK → users, nullable)
|     first_name, last_name, email, phone
|     cv_path, status (submitted|reviewing|accepted|rejected)
|     created_at

COMPLAINTS
| complaint_id (PK)

```

<code>user_id</code> (FK → <code>users</code>, nullable) <code>full_name</code>, <code>address</code>, <code>complaint_subcity</code>, <code>complaint_woreda</code> <code>type</code>, <code>status</code> (<code>assigning in progress resolved canceled</code>) <code>description</code> <code>photos</code> (JSON array), <code>videos</code> (JSON array), <code>audios</code> (JSON array) <code>concerned_staff_member</code>, <code>ref</code> (CPL-XXXXX) <code>created_at</code>
CONTACTS <code>id</code> (PK) <code>first_name</code>, <code>last_name</code>, <code>email</code>, <code>description</code> <code>photos</code> (JSON array) <code>status</code> (<code>pending resolved</code>) <code>created_at</code>
SERVICE_SATISFACTION <code>id</code> (PK) <code>gender</code>, <code>age</code>, <code>marital_status</code>, <code>education_level</code>, <code>employment_status</code> <code>district</code>, <code>visits</code>, <code>service_requested</code> (JSON array) <code>q1</code>, <code>q2</code>, <code>q3</code>, <code>q4</code>, <code>q5</code>, <code>q6</code>, <code>q7</code>, <code>q8</code>, <code>q9</code>, <code>q10</code>, <code>q11</code> (TEXT: <code>very_high high medium low very_low</code>) <code>additional_comments</code> <code>created_at</code>

7.3 Key Relationships

Relationship	Type	Notes
<code>admins</code> → <code>adminsettings</code>	1:1	Created on first settings access; <code>adminid</code> is both PK and FK
<code>admins</code> → <code>activity_logs</code>	1:N	Logs created for create/update/delete actions
<code>users</code> → <code>complaints</code>	1:N (optional)	<code>user_id</code> is nullable; anonymous complaints allowed
<code>users</code> → <code>applicants</code>	1:N (optional)	<code>user_id</code> is nullable; guest applications allowed
<code>news</code> → <code>news_translation</code>	1:1	Translation row may not exist; fallback to English
<code>events</code> → <code>eventstranslation</code>	1:1	<code>eventsid</code> in <code>events</code> maps to <code>event_id</code> in translation table
<code>vacancies</code> → <code>vacancy_translation</code>	1:1	Translation row may not exist
<code>vacancies</code> → <code>applicants</code>	1:N	Cascade delete on vacancy deletion

7.4 Class Diagram (Table Attributes)





8. API Specification

8.1 Base URL and Conventions

Environment	Base URL
Production	https://lidetasubcity.gov.et
Development	http://localhost:3000 (Vite proxies /api , /auth , /uploads to this)

All API endpoints are prefixed with [/api/](#) except authentication endpoints which also accept the [/auth/](#) prefix. All request and response bodies use [application/json](#) unless the endpoint accepts [multipart/form-data](#) for file uploads. All timestamps are ISO 8601 strings. All successful responses return HTTP 2xx. All error responses return [{ "error": "description string" }](#).

Authentication header format:

Authorization: Bearer <jwt_token>

8.2 Authentication Endpoints ([/auth/](#) or [/api/auth/](#))

POST [/auth/admin/login](#)

Authenticates an admin by username and password.

Request Body:

{ "username": "string", "password": "string" }
--

Response 200:

```
{
  "requires2FA": false,
  "token": "jwt_string",
  "admin": { "admin_id": "...", "username": "...", "role": "...", "email": "..." },
  "role": "string"
}
```

Response 401: { "error": "Invalid username or password" }

POST /auth/admin/email-lookup

Returns the email address for a given admin username. Used as a pre-step in 2FA.

Request Body: { "username": "string" } **Response 200:** { "email": "string" }

POST /auth/admin/me

Returns the authenticated admin's profile. Requires `Authorization: Bearer` .

Response 200: Admin object (no `password_hash`)

POST /api/auth/user/login

Authenticates a citizen user.

Request Body: { "email": "string", "password": "string" } **Response 200:** { "requires2FA": false, "token": "jwt_string", "user": { ... } } **Response 401:** { "error": "Invalid credentials" }

POST /api/auth/user/register

Registers a new citizen account and sends email verification OTP.

Request Body:

```
{ "first_name": "string", "last_name": "string", "email": "string", "phone": "string", "password": "string" }
```

Response 201:

```
{ "user": { ... }, "token": "jwt_string", "requiresVerification": true }
```

POST /api/auth/verify-otp

Verifies a 6-digit OTP code for both admin 2FA and user email verification.

Request Body: { "email": "string", "otp": "string", "entityType": "admin|user" } **Response 200:** { "token": "jwt_string", "admin|user": { ... } } **Response 400:** { "error": "Invalid or expired OTP" }

POST /api/auth/resend-otp

Resends the OTP. Rate-limited to 5 attempts per 10 minutes per email address.

Request Body: { "email": "string", "entityType": "admin|user", "purpose": "2faLogin|verifyemail|reset_password" } **Response 200:** { "message": "OTP sent" } **Response 429:** { "error": "Too many requests" }

POST `/api/auth/send-verification`

Sends email verification OTP to the currently authenticated user. Requires user JWT.

Response 200: `{ "message": "Verification email sent" }`

POST `/api/auth/verify-email`

Verifies the user's email address with OTP. Requires user JWT.

Request Body: `{ "otp": "string" }` **Response 200:** `{ "message": "Email verified" }`

POST `/api/auth/forgot-password`

Initiates password reset by sending OTP to the email address. Always returns 200 to prevent email enumeration.

Request Body: `{ "email": "string", "entityType": "admin|user" }` **Response 200:** `{ "message": "If this email exists, a code was sent" }`

POST `/api/auth/reset-password`

Resets the password after OTP verification.

Request Body: `{ "email": "string", "otp": "string", "newPassword": "string", "entityType": "admin|user" }` **Response 200:** `{ "message": "Password reset successfully" }` **Response 400:** `{ "error": "Invalid OTP" }`

8.3 User Endpoints (`/api/user/`)

All endpoints require `Authorization: Bearer` .

Method	Path	Description
GET	<code>/api/user/me</code>	Returns <code>{ id, firstname, lastname, email, phone }</code>
GET	<code>/api/user/dashboard</code>	Returns <code>{ complaints: [...], applications: [...] }</code>
PATCH	<code>/api/user/profile</code>	Updates name and/or phone. Body: <code>{ firstname, lastname, phone, currentPassword?, newPassword? }</code>

8.4 Admin Endpoints (`/api/admin/`)

All endpoints require `Authorization: Bearer` .

Method	Path	Description
POST	<code>/api/admin/update/profile-picture</code>	multipart/form-data with <code>profile_picture</code> field. Returns updated admin object.
DELETE	<code>/api/admin/delete/profile-picture</code>	Removes profile picture.
GET	<code>/api/admin/activities</code>	Returns last 20 activity log entries.
POST	<code>/api/admin/update/profile</code>	Updates personal info. Body: { <code>firstname</code> , <code>lastname</code> , <code>gender</code> , <code>residency</code> , <code>phone_number</code> , <code>email</code> }
POST	<code>/api/admin/update/admin-info</code>	Updates username. Only superadmin can change role.
POST	<code>/api/admin/update/password</code>	Changes password. Body: { <code>currentPassword</code> , <code>newPassword</code> }
GET	<code>/api/admin/settings</code>	Returns admin settings; creates defaults if none exist.
POST	<code>/api/admin/update/settings</code>	Upserts settings. Body: { <code>theme</code> , <code>font_size</code> , <code>language</code> }
POST	<code>/api/admin/upload</code>	Generic file upload. Returns { <code>name</code> , <code>path</code> , <code>size</code> , <code>mimetype</code> }
POST	<code>/api/admin/create-peer</code>	Creates admin with same role as caller.

8.5 Superadmin Endpoints (`/api/superadmin/`)

All endpoints require `Authorization: Bearer` . Returns HTTP 403 for non-superadmin tokens.

Method	Path	Description
GET	<code>/api/superadmin/admins</code>	Returns all admin accounts (array, no password_hash).
POST	<code>/api/superadmin/create-admin</code>	Creates new admin. Body: all admin fields + <code>role</code> .
POST	<code>/api/superadmin/update-admin/:id</code>	Updates admin by <code>admin_id</code> .
DELETE	<code>/api/superadmin/delete-admin/:id</code>	Deletes admin. Cannot delete own account.
GET	<code>/api/superadmin/overview</code>	Returns { <code>totalComplaints</code> , <code>resolvedComplaints</code> , <code>pendingApplications</code> , <code>activeEvents</code> }
GET	<code>/api/superadmin/vacancy-applications</code>	Returns all applicants with vacancy info and category stats.

8.6 News Endpoints (`/api/news/`)

GET `/api/news`

Returns all news articles with translations.

Response 200:

```
[
  {
    "id": 1,
    "title": "string",
    "short_description": "string",
    "description": "string",
    "category": "string",
    "photo": { "path": "string", "name": "string" },
    "formatted_date": "Jan 01, 2026",
    "amh": { "title": "...", "short_description": "...", "description": "...", "category": "..." },
    "orm": { "title": "...", "short_description": "...", "description": "...", "category": "..." }
  }
]
```

GET `/api/news/admin`

Same as public GET. Requires admin JWT.

POST `/api/news/admin`

Creates a news article. Requires admin JWT.

Request Body:

```
{
  "title": "string", "description": "string", "category": "string",
  "shortDescription": "string",
  "photo": { "path": "string", "name": "string" },
  "amh": { "title": "...", "short_description": "...", "description": "...", "category": "..." },
  "orm": { "title": "...", "short_description": "...", "description": "...", "category": "..." }
}
```

Response 201: Created news object.

PUT `/api/news/admin`

Updates a news article by `id`. Requires admin JWT. Same body as POST plus `id` field.

DELETE `/api/news/admin/:id`

Deletes a news article and its translation. Requires admin JWT.

8.7 Events Endpoints (`/api/events/`)

Method	Path	Auth	Description
GET	<code>/api/events</code>	None	All events with translations and formatted date.
GET	<code>/api/events/admin</code>	Admin JWT	Same as public GET.
POST	<code>/api/events/admin</code>	Admin JWT	Create event. Body wrapped in <code>formData</code> object.
PUT	<code>/api/events/admin</code>	Admin JWT	Update event. Body wrapped in <code>formData</code> object.
DELETE	<code>/api/events/admin/:id</code>	Admin JWT	Delete event and translation.

Event object shape:

```
{
  "events_id": 1, "title": "string", "description": "string",
  "location": "string", "start_date": "ISO date", "end_date": "ISO date",
  "status": "upcoming|pending|complete|canceled",
  "photos": [{ "path": "string", "name": "string" }],
  "start_date_short": "Mon. Jan, 01 2026",
  "amh": { "title": "...", "description": "...", "location": "..." },
  "orm": { "title": "...", "description": "...", "location": "..." }
}
```


8.8 Vacancies Endpoints (`/api/vacancies/`)

Method	Path	Auth	Description
GET	<code>/api/vacancies</code>	None	All vacancies with translations.
POST	<code>/api/vacancies/upload-cv</code>	None	Upload CV file. Returns <code>{ path: "/uploads/cvs/filename" }</code>
POST	<code>/api/vacancies/applicants</code>	None	Submit job application. Returns <code>{ ...applicant, ref: "APP-XXXXXX" }</code>
GET	<code>/api/vacancies/admin</code>	Admin JWT	All vacancies for admin view.
POST	<code>/api/vacancies/admin</code>	Admin JWT	Create vacancy.
PUT	<code>/api/vacancies/admin</code>	Admin JWT	Update vacancy.
DELETE	<code>/api/vacancies/admin/:id</code>	Admin JWT	Delete vacancy, applicants, and translation.
GET	<code>/api/vacancies/applicants/admin</code>	Admin/Vacancy JWT	All applicants with vacancy info.
GET	<code>/api/vacancies/applicants/admin/:id</code>	Admin/Vacancy JWT	Single applicant detail.
PUT	<code>/api/vacancies/applicants/admin/:id</code>	Admin/Vacancy JWT	Update applicant (status, CV).

8.9 Complaints Endpoints (`/api/complaints/`)

Method	Path	Auth	Description
POST	<code>/api/complaints</code>	None	Submit complaint. Returns <code>{ complaint_id, ref: "CPL-XXXXXX" }</code>
GET	<code>/api/complaints/types</code>	None	Returns list of complaint types.
GET	<code>/api/complaints/admin</code>	Admin JWT	Returns <code>{ complaints[], counts: {total, pending, resolved}, stats[] }</code>
POST	<code>/api/complaints/admin</code>	Admin JWT	Admin creates complaint record.
POST	<code>/api/complaints/admin/update</code>	Admin JWT	Update complaint (status, notes, etc.).
DELETE	<code>/api/complaints/admin/:id</code>	Admin JWT	Delete complaint by <code>complaint_id</code> .

8.10 Contacts Endpoints (`/api/contacts/`)

Method	Path	Auth	Description
POST	<code>/api/contacts</code>	None	Submit contact message.
POST	<code>/api/contacts/satisfaction</code>	None	Submit satisfaction survey.
GET	<code>/api/contacts/admin</code>	Admin JWT	View pending contact messages.
PUT	<code>/api/contacts/admin/:id/resolve</code>	Admin JWT	Mark contact as resolved.
GET	<code>/api/contacts/satisfaction/stats</code>	Admin JWT	30-day satisfaction stats: averages + daily counts.

8.11 Static File Endpoints

Path	Description
GET /uploads/photos/*	Publicly accessible uploaded photos
GET /uploads/videos/*	Publicly accessible uploaded videos
GET /uploads/audios/*	Publicly accessible uploaded audio files
GET /uploads/cvs/*	Uploaded CVs — publicly accessible by direct URL
GET /uploads/admin_profiles/*	Admin profile pictures — publicly accessible
GET /health	Health check: { ok: true, message, timestamp, env }

9. Real-Time Communication

9.1 Current Implementation

The Lideta Sub-City system does **not** implement any real-time communication mechanism in the current version. All communication between the client and server is strictly request-response over HTTP/HTTPS using the REST API defined in Section 8.

There are no WebSocket connections, no Server-Sent Events (SSE), no long-polling mechanisms, and no third-party push notification services (Firebase Cloud Messaging, Pusher, etc.) in use.

Security implication for penetration testers: There are no WebSocket upgrade handshake paths to test. All `/api/` and `/auth/` paths are standard HTTP endpoints; no `Upgrade: websocket` header handling exists.

9.2 Future Considerations

If real-time features are added in future versions, the following would be in scope for security review:

- **Complaint status push notifications** — Notifying a citizen when their complaint status changes without requiring a page refresh.
- **Admin live notifications** — Alerting admin staff of new complaint submissions or new contact messages in real time.
- **Proposed technology:** Socket.io (WebSocket with HTTP long-poll fallback) integrated into the existing Express server.
- **Security considerations for future implementation:** WebSocket connections would require JWT-based handshake authentication, per-connection rate limiting, input sanitization for all message payloads, and protection against Cross-Site WebSocket Hijacking (CSWSH).

10. Security Requirements

10.1 Authentication Security

- **SR-001:** All admin and citizen passwords shall be hashed using `bcryptjs` with a minimum work factor of 10 salt rounds before storage.
- **SR-002:** JWT tokens shall be signed with a minimum 256-bit random secret (`JWT_SECRET`) stored in environment variables. The secret shall never be committed to source control.
- **SR-003:** Admin JWTs shall expire after 7 days. Citizen JWTs shall expire after 30 days.
- **SR-004:** The token verification middleware (`authenticateToken`, `authenticateUser`) shall reject tokens that are expired, malformed, or signed with a different secret.

- **SR-005:** The password reset flow shall not confirm whether a given email address exists in the system (always returns HTTP 200 with a generic message) to prevent user enumeration.
- **SR-006:** Password reset and email verification OTP codes are single-use, 6-digit numeric codes generated and verified by the OTP email service.
- **SR-007:** OTP resend operations are rate-limited to 5 attempts per 10-minute sliding window per email address, enforced server-side.

10.2 Password Policy

- **SR-008:** All passwords (registration, admin creation, password change, password reset) shall meet the following requirements: minimum 8 characters, at least one uppercase letter, at least one lowercase letter, at least one digit, at least one special character.
- **SR-009:** Password strength is validated both client-side (with visual feedback) and enforced server-side. Client-side validation is a UX enhancement only and does not constitute a security boundary.

10.3 Transport Security

- **SR-010:** All communication in production shall be over HTTPS/TLS. HTTP requests shall be redirected to HTTPS by the Plesk web server configuration.
- **SR-011:** The database connection uses SSL with `{ rejectUnauthorized: false }` in the current configuration. This should be upgraded to certificate pinning in a hardened production environment.
- **SR-012:** The OTP email service API calls use HTTPS by default.

10.4 Input Validation

- **SR-013:** All API endpoint input shall be treated as untrusted. Database queries use the `postgres` tagged template literal driver which parameterizes all values, preventing SQL injection.
- **SR-014:** File uploads are validated by MIME type (images, video, audio, PDF, Word documents only) and limited to 50MB per file.
- **SR-015:** CV file uploads are restricted to `application/pdf` only at the application layer (in addition to the general multer MIME type filter).
- **SR-016:** Ethiopian phone number inputs are validated server-side and client-side against the pattern `/^(\+251(9|7)\d{8}|0(9|7)\d{8})$/`.
- **SR-017:** File names are sanitized on upload: non-alphanumeric characters are replaced with underscores and a random suffix is appended to prevent path traversal and filename collision.

10.5 CORS Policy

- **SR-018:** The CORS policy allows requests only from explicitly whitelisted origins: `localhost` (development), `https://lideta-official.vercel.app`, `https://lidetasubcity.gov.et`, `https://www.lidetasubcity.gov.et`, any `*.run.app` origin, and the `CLIENT_ORIGIN` environment variable.
- **SR-019:** `credentials: true` is set in the CORS configuration, allowing cookies and Authorization headers from whitelisted origins.
- **SR-020:** Requests from non-whitelisted origins are rejected with an HTTP CORS error.

10.6 File Upload Security

- **SR-021:** All uploaded files are stored on the server filesystem at `client/public/uploads/`. This directory is served as a static endpoint (`GET /uploads/*`) and files are publicly accessible by direct URL.

- **SR-022:** CV files (`/uploads/cvs/*`) are publicly accessible by direct URL. Access should be restricted to authenticated admin users in a future hardening iteration.
- **SR-023:** Uploaded filenames include a timestamp and random 9-digit suffix to prevent predictable URL enumeration.
- **SR-024:** No executable file types (`.js`, `.php`, `.sh`, `.exe`) are permitted by the file filter.

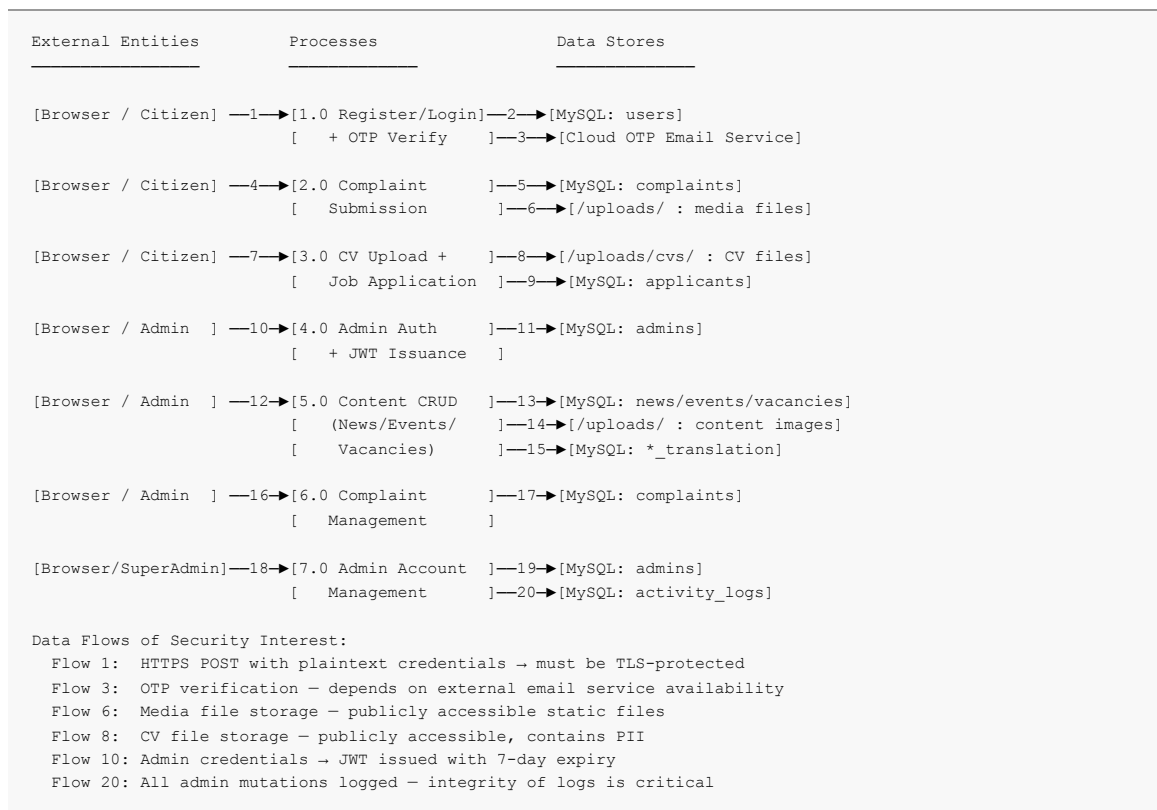
10.7 Session and Token Security

- **SR-025:** Both admin and citizen JWTs are stored in the browser's `localStorage`. This makes them accessible to JavaScript running on the page.
- **SR-026:** No `HttpOnly` cookie-based token storage is currently implemented. `localStorage` storage is susceptible to XSS-based token theft. Migrating to `HttpOnly` cookies is recommended for future versions.
- **SR-027:** No explicit token revocation mechanism (blocklist) is implemented. Token invalidation relies solely on expiry.

10.8 Activity Logging

- **SR-028:** All create, update, and delete operations performed by admin accounts shall be recorded in the `activitylogs` table with: *adminid*, *username*, *action*, *entitytype*, *entitytitle*, *changed fields*, and *timestamp*.
- **SR-029:** Activity logs are read-only through the API (no delete endpoint exists for activity logs).

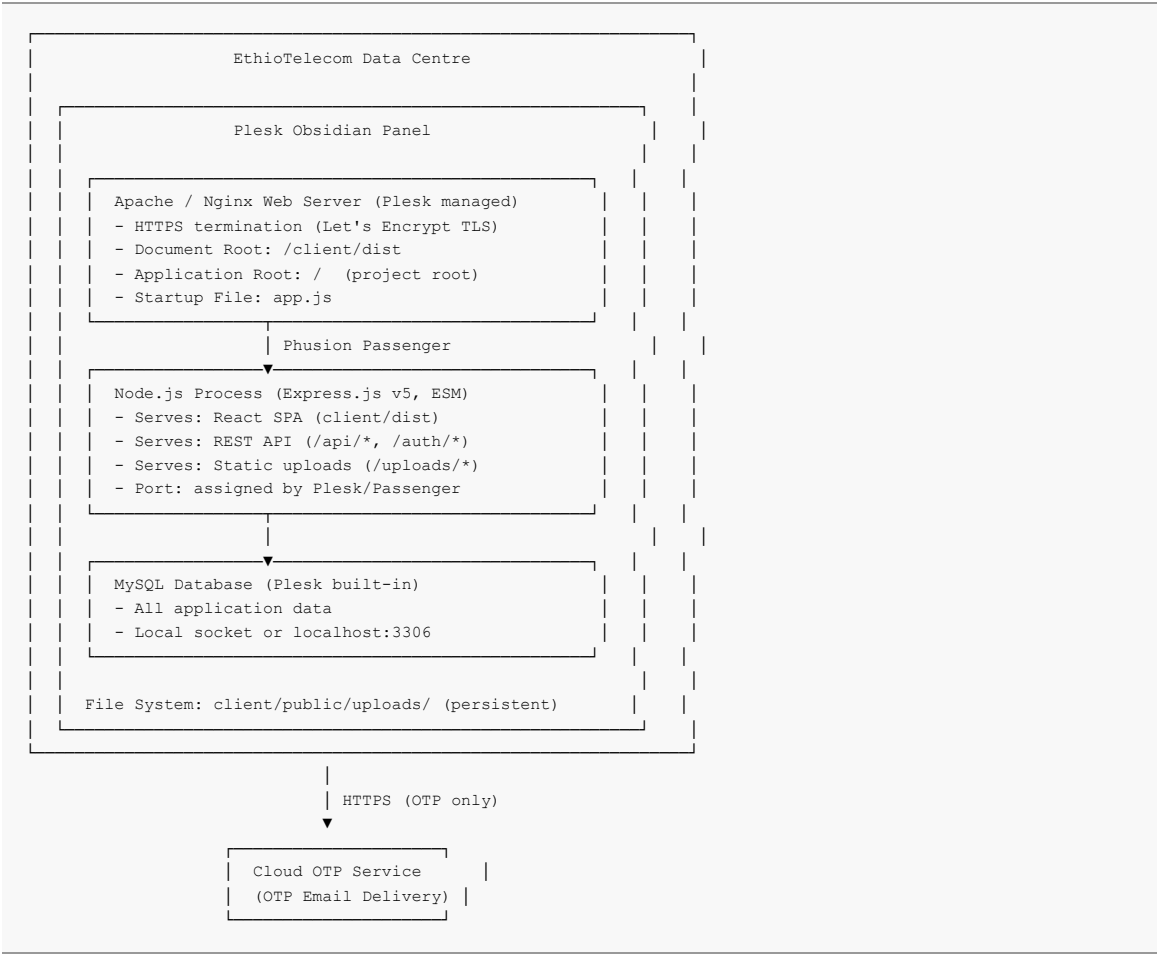
10.9 Data Flow Security (DFD Level 1)



11. Deployment Architecture

11.1 Production Environment

The system is deployed on **EthioTelecom Plesk Obsidian** shared web hosting using the **Phusion Passenger** Node.js application server module.



11.2 Plesk Configuration

Setting	Value
Application Type	Node.js (Phusion Passenger)
Application Root	<code>/</code> (project root)
Document Root	<code>/client/dist</code>
Startup File	<code>app.js</code>
Node.js Version	18+
Application Mode	Production
Environment Variables	Set via Plesk Node.js panel: <code>DATABASEURL</code> , <code>JWT_SECRET</code> , <code>OTPSERVICEURL</code> , <code>OTPSERVICEKEY</code> , <code>SERVERPORT</code> , <code>CLIENT_ORIGIN</code> , <code>NODE_ENV=production</code>

app.js (root entry point) is a one-line ESM import that starts the Express server:

```
import './server/server.js';
```

This file exists because Plesk's Phusion Passenger requires a startup file at the configured path.

11.3 URL Routing in Production

Plesk's web server is configured to proxy all requests to the Node.js Passenger process. The Express server handles routing internally:

Request Pattern	Handler
<code>/api/*</code>	Express API routes
<code>/auth/*</code>	Express auth routes
<code>/uploads/*</code>	Express static file serving
<code>/health</code>	Express health check
All others	SPA fallback → <code>client/dist/index.html</code>

11.4 Static File Serving

The React SPA is built by running `npm run build:client` from the project root, which executes `vite build` in the `client/` directory. The output is placed in `client/dist/`. Express serves this directory as static files.

User-uploaded files (images, videos, audio, CVs, profile pictures) are stored at `client/public/uploads/` and served via `app.use('/uploads', express.static(...))`.

11.5 TLS/HTTPS

TLS termination is handled by Plesk's Apache/Nginx layer using Let's Encrypt certificates. The Node.js application does not handle TLS directly. All HTTP traffic is redirected to HTTPS by the Plesk web server configuration.

11.6 Environment Variables

Variable	Required	Description
<code>DATABASE_URL</code>	Yes	Full MySQL connection string
<code>JWT_SECRET</code>	Yes	HS256 signing secret for all JWTs
<code>SERVER_PORT</code>	Yes	Port assigned by Plesk/Passenger
<code>OTPSERVICEURL</code>	Yes	Cloud OTP email service endpoint URL
<code>OTPSERVICEKEY</code>	Yes	Cloud OTP email service authentication key
<code>CLIENT_ORIGIN</code>	Recommended	Additional allowed CORS origin
<code>NODE_ENV</code>	Recommended	Set to <code>production</code> in live environment

11.7 Development Environment

For local development, the system uses a split setup:

Component	Command	Port
Express API server	<code>npm run dev</code> (from project root)	3000
React Vite dev server	<code>npm run dev</code> (from <code>client/</code>)	5173

The Vite dev server proxies the following paths to `localhost:3000`:

```

/api    → http://localhost:3000
/auth   → http://localhost:3000
/uploads → http://localhost:3000

```

This allows the React SPA at port 5173 to call API endpoints without CORS issues during development. In production, there is no Vite dev server; Express serves the built SPA directly.

If `DATABASE_URL` is not set in development, the server falls back to an in-memory mock database seeded from the static JSON files in `client/src/data/`. Default credentials in mock mode: all accounts use password `admin123`.

12. Appendices

Appendix A — Technology Stack Summary

Layer	Technology	Version
Front-End Framework	React	19.2.0
Front-End Build Tool	Vite	7.2.4
Front-End Routing	React Router	7.9.6
CSS Framework	Tailwind CSS	4.1.17
Animation	Framer Motion	13.1.0
Charts	Recharts	3.5.1
Icons	SVG (vite-plugin-svg)	4.5.0
Back-End Framework	Express.js	5.2.1
Runtime	Node.js	18+
Module System	ESM ("type": "module")	—
Database	MySQL (Plesk built-in)	—
DB Driver	<u>postgres</u> (tagged template)	3.4.7
Authentication	JSON Web Tokens (jsonwebtoken)	9.0.3
Password Hashing	bcryptjs	3.0.3
File Uploads	Multer	2.0.2
OTP Email	Cloud Email Delivery Client	—
CORS	cors (npm)	2.8.5
Deployment	Plesk / Phusion Passenger	—
TLS	Let's Encrypt (via Plesk)	—

Appendix B — Complete Route Tree

PUBLIC ROUTES (layout: <Home />)	
└─ /	→ HomePage
└─ /about_us	→ AboutUs
└─ /departments	→ Departments
└─ /departments/:id	→ DepartmentDetails
└─ /news	→ News
└─ /news/:id	→ NewsDetails
└─ /events	→ Events
└─ /events/:id	→ EventDetails
└─ /vacancy	→ Vacancy
└─ (legacy: /vaccancy	→ redirect /vacancy)
└─ /vacancy/:id	→ VacancyDetails
└─ /complaints	→ Complaints
└─ (legacy: /complaints	→ redirect /complaints)
└─ /contacts	→ Contacts
CITIZEN AUTH	
└─ /account/auth	→ UserAuth (login / register / OTP / forgot-password)
└─ /account	→ UserDashboard
ADMIN AUTH	
└─ /auth/login	→ Login (admin only)
ADMIN PANEL (layout: <Admin />, provides adminContext)	
└─ /admin	→ AdminHome [role: admin]
└─ /admin/complaints	→ AdminComplaints [role: admin, complaint_admin]
└─ (legacy: /admin/complaints	→ redirect)
└─ /admin/events	→ AdminEvent [role: admin, event_admin]
└─ /admin/news	→ AdminNews [role: admin, news_admin]
└─ /admin/vacancy	→ AdminVacancy [role: admin, vacancy_admin]
└─ /admin/profile	→ AdminProfile [role: all admin roles]
SUPERADMIN PANEL (layout: <SuperAdminLayout />)	
└─ /superadmin	→ redirect /superadmin/home
└─ /superadmin/home	→ SuperAdminHome
└─ /superadmin/profile	→ SuperAdminProfile

Appendix C — File Upload Paths

Upload Type	Field Name	Storage Path	Public URL
News/Event photos	photo / image	client/public/uploads/photos/	/uploads/photos/
Complaint photos	photo	client/public/uploads/photos/	/uploads/photos/
Complaint videos	video	client/public/uploads/videos/	/uploads/videos/
Complaint audio	audio	client/public/uploads/audios/	/uploads/audios/
Admin profile pics	profilepicture	client/public/uploads/adminprofiles/	/uploads/admin_profiles/
Applicant CVs	cv	client/public/uploads/cvs/	/uploads/cvs/.pdf

All uploaded files are publicly accessible by direct URL. No authentication is required to download any uploaded file.

Appendix D — Environment Variable Reference


```
# Database
DATABASE_URL=mysql://user:password@localhost:3306/lideta_db

# Security
JWT_SECRET=<minimum-256-bit-random-string>

# Server
SERVER_PORT=<assigned-by-plesk>
NODE_ENV=production

# OTP Email Service
OTP_SERVICE_URL=https://<otp-service-endpoint>
OTP_SERVICE_KEY=<otp-service-api-key>

# CORS
CLIENT_ORIGIN=https://lidetasubcity.gov.et
```

Appendix E — Admin Role Default Landing Paths

Role	Default Path After Login
superadmin	/superadmin/home
admin	/admin
complaint_admin	/admin/complaints
event_admin	/admin/events
news_admin	/admin/news
vacancy_admin	/admin/vacancy

Appendix F — Satisfaction Survey Question Reference

The service satisfaction survey collects responses to 11 questions (q1–q11), each scored on a 5-point Likert scale:

veryhigh (5), high (4), medium (3), low (2), very low (1) .

Question ID	Subject
q1	Overall satisfaction with the service
q2	Staff courtesy and professionalism
q3	Speed and timeliness of service
q4	Clarity of information provided
q5	Ease of accessing the service
q6	Physical environment / office conditions
q7	Resolution of complaints or issues
q8	Transparency of the process
q9	Availability of staff when needed
q10	Digital/online service satisfaction
q11	Likelihood of recommending the service

Aggregate statistics are calculated server-side and exposed at `GET /api/contacts/satisfaction/stats` as 30-day averages per question plus daily response volume.

Appendix G — Known Limitations and Future Recommendations

Item	Current State	Recommendation
JWT storage	<code>localStorage</code> (XSS-susceptible)	Migrate to <code>HttpOnly</code> + <code>SameSite=Strict</code> cookies
CV file access	Publicly accessible by direct URL	Restrict to authenticated admin users via signed URLs or middleware auth check
2FA flag	<code>requires2FA: false</code> hardcoded for all accounts	Implement per-account 2FA toggle in admin settings
Real-time notifications	Not implemented	Add Socket.io for live complaint status updates
Horizontal scaling	Single process	Externalize session/token store if scaling to multiple instances
Activity log integrity	No tamper protection	Add append-only database permission or external log export
File type validation	MIME type only (spoofable)	Add magic byte / file signature validation for uploads
SSL certificate pinning	<code>rejectUnauthorized: false</code> for DB	Enable full certificate validation in production

End of SRS.md — Lideta Sub-City Official Website and Administration System v1.0.0